

UNIVERSIDADE (UNIRUY)

Curso: CIÊNCIA DA COMPUTAÇÃO

Disciplina: Programação para Dispositivos Móveis

Título: ClimaMobile — Aplicativo móvel para consulta de clima usando OpenWeatherMap

Autor: LUCAS BRENDON ALVES DOS SANTOS CONCEICAO **Matrícula:** 202402533843

Orientador: ENDERSON SANTOS

Local: SALVADOR

Data: 01 de junho de 2026

RESUMO

Este relatório descreve o desenvolvimento do ClimaMobile, um aplicativo móvel multiplataforma (Expo/React Native) para consulta do clima atual e previsão de curto prazo utilizando a API OpenWeatherMap. O trabalho apresenta objetivos, metodologia de implementação, arquitetura do sistema, principais componentes, testes realizados, resultados obtidos e discussões sobre limitações e melhorias futuras. O aplicativo inclui busca por cidade, geolocalização do dispositivo, armazenamento em cache com `AsyncStorage` para uso offline e exibição de previsões diárias resumidas.

PALAVRAS-CHAVE: Clima; OpenWeatherMap; React Native; Expo; Aplicativo móvel; Cache offline.

ABSTRACT

This report describes the development of ClimaMobile, a cross-platform mobile application (Expo/React Native) for retrieving current weather and short-term forecasts using the OpenWeatherMap API. It covers objectives, implementation methodology, system architecture, main components, tests performed, results and discussion about limitations and future enhancements. The app supports city search, device geolocation, local cache using `AsyncStorage`, and simplified daily forecast display.

KEYWORDS: Weather; OpenWeatherMap; React Native; Expo; Mobile app; Offline cache.

SUMÁRIO

- RESUMO
- INTRODUÇÃO
- OBJETIVOS
- METODOLOGIA
- ARQUITETURA E IMPLEMENTAÇÃO
 - Estrutura do projeto
 - Serviços e integração com API
 - Interface de usuário
 - Persistência e cache
- TESTES E VALIDAÇÃO
- RESULTADOS
- DISCUSSÃO
- CONCLUSÃO
- REFERÊNCIAS
- ANEXOS

1. INTRODUÇÃO

Aplicativos de previsão do tempo são amplamente usados para planejamento diário e decisões relacionadas a mobilidade, agricultura e eventos ao ar livre. Este trabalho tem como objetivo projetar e implementar um aplicativo móvel simples e funcional que demonstre integração com uma API externa de meteorologia, uso de permissões de localização, tratamento básico de falhas e persistência local para permitir uso em condições offline.

O desenvolvimento foi realizado com Expo e React Native, aproveitando recursos nativos simplificados pelo Expo, e a API utilizada foi a OpenWeatherMap.

2. OBJETIVOS

- Desenvolver um aplicativo móvel multiplataforma que exiba clima atual e previsão por cidade e por coordenadas.
- Implementar leitura de localização do dispositivo e pedir permissões de forma adequada.
- Implementar cache local para permitir visualização de dados quando não houver conectividade.
- Fornecer uma interface clara e responsiva para usuários mobile.

3. METODOLOGIA

A metodologia adotada foi de desenvolvimento incremental com as seguintes etapas:

- Levantamento de requisitos mínimos (busca por cidade, geolocalização, cache e previsão).
- Configuração do projeto com Expo e dependências (ver `package.json`).
- Implementação dos serviços de integração com a API usando `axios`.
- Implementação da tela principal (`buscacidade.js`) com hooks React para gerenciar estado e efeitos.
- Testes manuais em simulador/dispositivo e verificação de cenários offline.

4. ARQUITETURA E IMPLEMENTAÇÃO

4.1 Estrutura do projeto

- Arquivo principal de entrada: `app/index.tsx` reexporta a tela de busca.
- Tela principal: `buscacidade.js` (componentes funcionais, hooks e estilos).
- Serviços:
 - `services/api.ts` — encapsula chamadas à OpenWeatherMap (`/weather` e `/forecast`).
 - `services/location.ts` — gerencia permissões e leitura de localização via `expo-location`.
- Configurações: `package.json`, [README.md](#).

4.2 Serviços e integração com API

As chamadas ao serviço Web são feitas em `services/api.ts` usando `axios`:

- `getWeatherByCity(city: string)` — obtém clima atual por nome da cidade.
- `getWeatherByCoords(lat: number, lon: number)` — obtém clima atual por coordenadas.
- `getForecastByCoords(lat: number, lon: number)` — obtém previsão em 3h (endpoint `/forecast`).

Observação: A chave de API (`API_KEY`) está presente no arquivo `services/api.ts`. Em cenários de produção, recomenda-se mover a chave para um backend ou usar variáveis de ambiente seguras.

4.3 Interface de usuário

A interface é implementada em `buscarcidade.js`. Componentes e comportamentos principais:

- Campo de entrada para digitar a cidade (`TextInput`).
- Botões: `Buscar`, `Localização Atual`.
- Atalhos para capitais/estados (`ESTADOS_SUGERIDOS`) com `TouchableOpacity`.
- Cartões que exibem cidade, temperatura atual arredondada, descrição do tempo, sensação térmica, umidade, vento e chuva quando presente.
- Exibição de previsão diária composta por amostragem da lista retornada pela API (`formatForecast`).
- Indicador de carregamento (`ActivityIndicator`) e mensagens de erro/estado offline.

4.4 Persistência e cache

- O app usa `@react-native-async-storage/async-storage` para salvar o último conjunto de dados consultado sob a chave `lastWeatherCache`.
- Em caso de falha nas requisições, o app tenta carregar o cache e informa o usuário com uma mensagem.

5. TESTES E VALIDAÇÃO

Testes realizados manualmente:

- Busca por cidade válida → verificado preenchimento correto da UI.
- Uso de localização → verificado fluxo de permissão e recuperação de coordenadas.
- Falha de rede simulada → verificado carregamento de cache salvo e exibição de mensagem offline.

Casos de teste formais sugeridos (para implementação futura):

- CT01: Busca válida retorna status 200 e dados obrigatórios (nome, main.temp, weather[]).
- CT02: Busca inválida (cidade inexistente) → tratamento de erro e mensagem ao usuário.
- CT03: Permissão de localização negada → app informa necessidade de ativar GPS.

- CT04: Offline com cache disponível → mostra dados do cache e marca como offline.

6. RESULTADOS

O aplicativo alcançou os objetivos principais: consulta por cidade, geolocalização e previsão, com cache para uso offline. A interface é simples e adequada para demonstração acadêmica.

7. DISCUSSÃO

Limitações:

- Exposição da chave de API no cliente é um risco de segurança.
- Não há testes automatizados integrado ao repositório.
- Tratamento de erros e mensagens poderiam ser mais detalhados e abrangentes.
- A previsão diária é obtida por amostragem do endpoint de 3h; uma alternativa é usar o endpoint `onecall` (quando disponível) para dados agregados.

Melhorias propostas:

- Implementar backend simples para proxy da API e ocultação da chave.
- Adicionar testes unitários para componentes e mocks de rede.
- Implementar CI para builds e testes (GitHub Actions, por exemplo).
- Melhorar internacionalização e acessibilidade.

8. CONCLUSÃO

O ClimaMobile é um MVP funcional que demonstra conceitos essenciais do desenvolvimento mobile com integração de APIs externas, gerenciamento de permissões, persistência local e tratamento de falhas. O projeto é apropriado como entrega acadêmica e possui caminhos claros para evolução técnica.

9. REFERÊNCIAS

- OPENWEATHERMAP. OpenWeatherMap API. Disponível em: <https://openweathermap.org/> . Acesso em: 03 jun. 2026.
- EXPO DOCUMENTATION. Expo Docs. Disponível em: <https://docs.expo.dev/> . Acesso em: 03 jun. 2026.
- REACT NATIVE. React Native Docs. Disponível em: <https://reactnative.dev/> . Acesso em: 03 jun. 2026.
- AXIOS. Axios GitHub. Disponível em: <https://github.com/axios/axios> . Acesso em: 03 jun. 2026.
- Async Storage. @react-native-async-storage/async-storage. Disponível em: <https://react-native-async-storage.github.io/async-storage/docs/install/> . Acesso em: 03 jun. 2026.

10. ANEXOS

10.1 Trechos de código relevantes

- Serviço API: `services/api.ts` (chave `API\_KEY` presente)
- Tela principal: `buscardade.js` (busca, localização, cache e UI)
- Serviço de localização: `services/location.ts` (permissões e leitura de posição)
- Configuração de dependências: `package.json`

10.2 Observações operacionais

- Para executar o projeto localmente:

```
```bash
npm install
npx expo start
```
```

- Recomendação de uso: testar em dispositivo físico para funcionalidade de GPS; em simuladores, configurar localização manualmente.